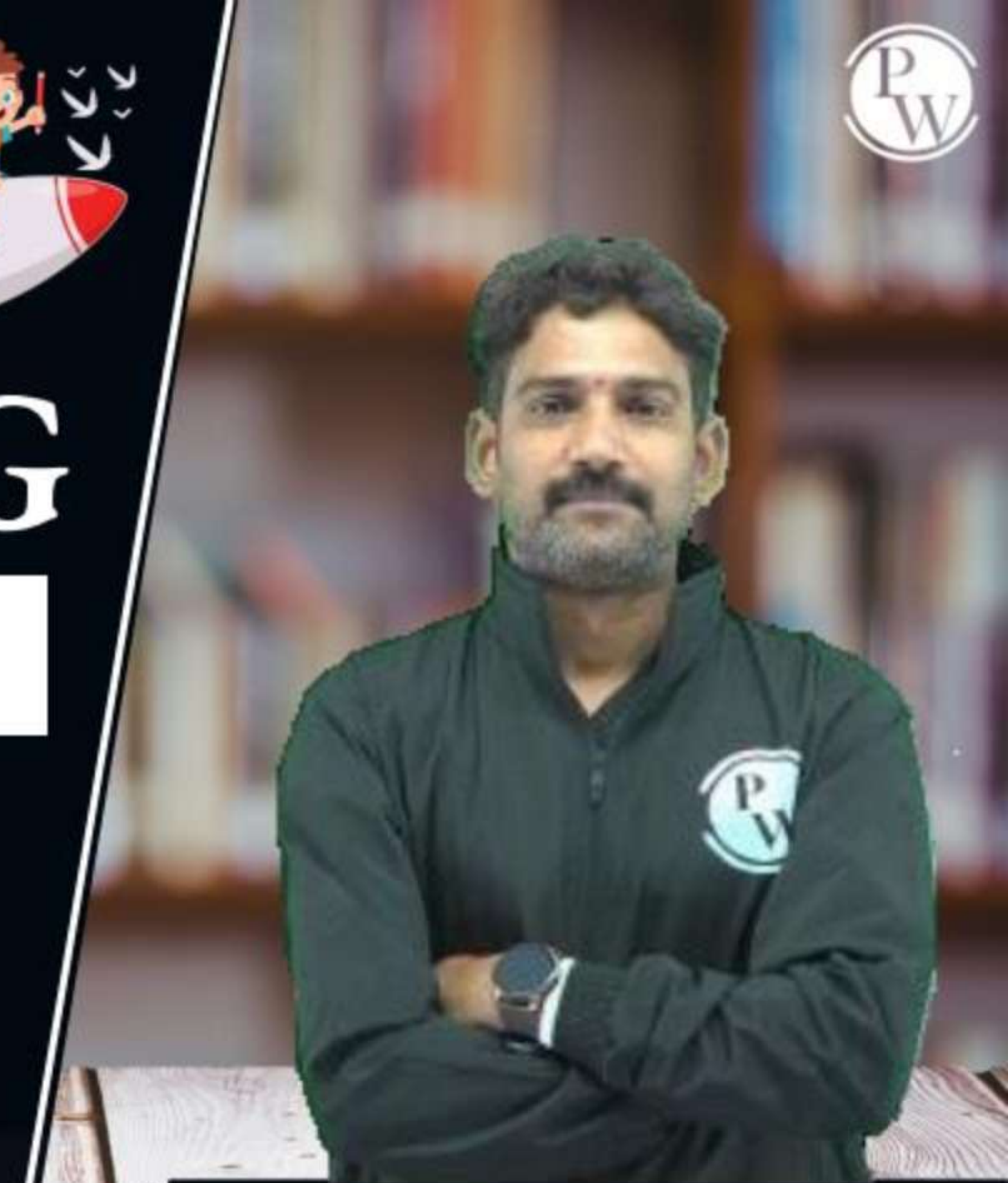
CS & IT ENGINEERING

'C' Programming

Pointers & Arrays



Lecture No.- 05

By- Satya sir

Recap of Previous Lecture

- String handling functions
- 2-D arrays
  - Declaration
  - Initialization
  - i/o 2-D array Elements
  - address of 2-D array Elements
    - RMO
    - CMO

Topics to be Covered

**Pointers and arrays

- Pointer Vs array
- Pointers & 1-D arrays
- Pointers & 2-D arrays
- 3-D arrays*

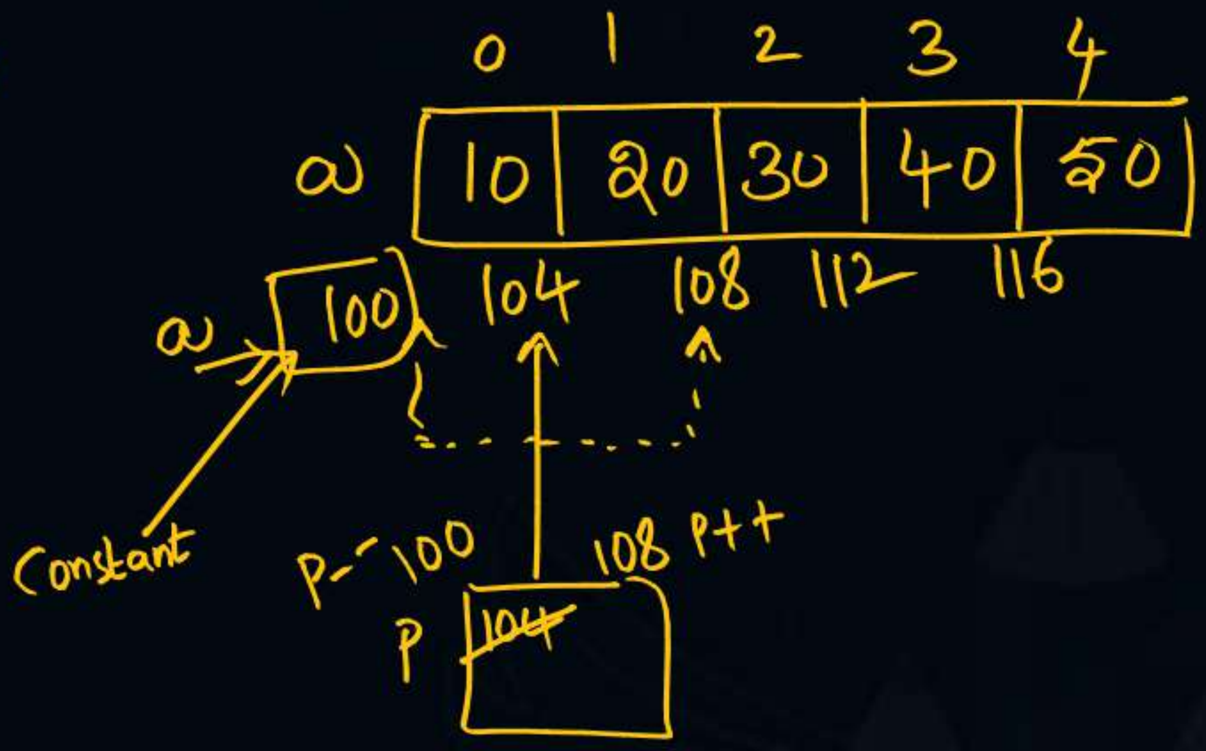
Pointer VS Array

```
int a[5]={10,20,30,40,50};

int *p = &a[1];

a++;      a--; // Error
a=a+1;    a=a-1;

p++;   p--;   // valid
```



NOTE: Pointer is Variable, while array is Constant.

Topic : Pointers and Arrays

```
int x[3]={10,20,30}, *P;

P= x (or) &x[0] ;

P++ / P-- ; valid
x++ / x-- ; Invalid

P=NULL;  // Valid
x=NULL;  // Invalid
```

NOTE 2: Assignment with NULL is valid for Pointer but not an Array.

```
int x[3];
:
x[3]={10,20,30};
// Invalid
```

```
x[0]=10;
x[1]=20;
x[2]=30;
Valid
```

```
int *P= &x[0];
(OR)
int *P;
:
P= &x[0];
```

NOTE : 3 Initialization of array Elements together can be done only at the time of declaration. But, Pointer can be Initialized any time.

Topic : Pointers and Arrays

&s ⇒ whole array

&s[0] ⇒ s[0] address

&s[3] ⇒ s[3] address

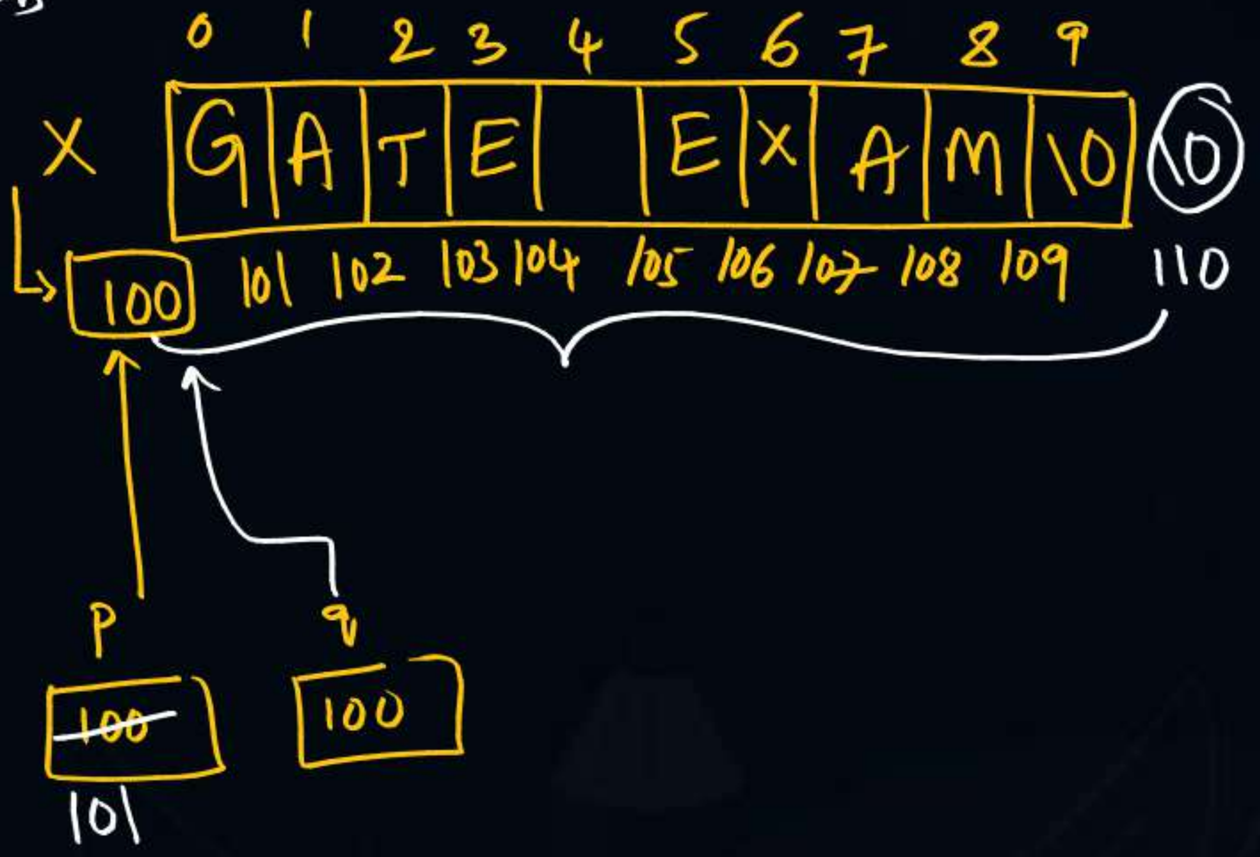
Ex:

```
char s[10] = "GATE EXAM";

char *p = s;   (*q)[10] = &s;   → whole array
```

One element of array

```
printf("%s", s);  // GATE EXAM
printf("%s", p);  // GATE EXAM
printf("%s", q);  // GATE EXAM
printf("%u", p);  // 100
printf("%u", q);  // 100
printf("%u", s);  // 100
```




```
p++;  // p = p+1*1 = 100+1*1 = 101
q++;  // q = q+1*10 = 100+10 = 110
printf("%u %s", p, p);  // 101 ATE EXAM
printf("%u %s", q, q);  // 110 Not Printed anything
```

Topic : Pointers and Arrays

Pointers with 1-D array

Let 16-bit Processor, 1 int = 2 Bytes, B = 100

```
int x[5] = {11, 22, 33, 44, 55};

int *P = x;
```

	0	1	2	3	4
x	11	22	33	44	55
	100	102	104	106	108
	x = x+0	x+1	x+2	x+3	x+4
	P+0	P+1	P+2	P+3	P+4

P: 100

when Assigned Pointer to base address of an array

$$x+0 == P+0 == \&x[0]$$

$$x+1 == P+1 == \&x[1]$$

$$\vdots \qquad \vdots \qquad \vdots$$

$$x+3 == P+3 == \&x[3]$$

$$\boxed{x+i == P+i == \&x[i]}$$

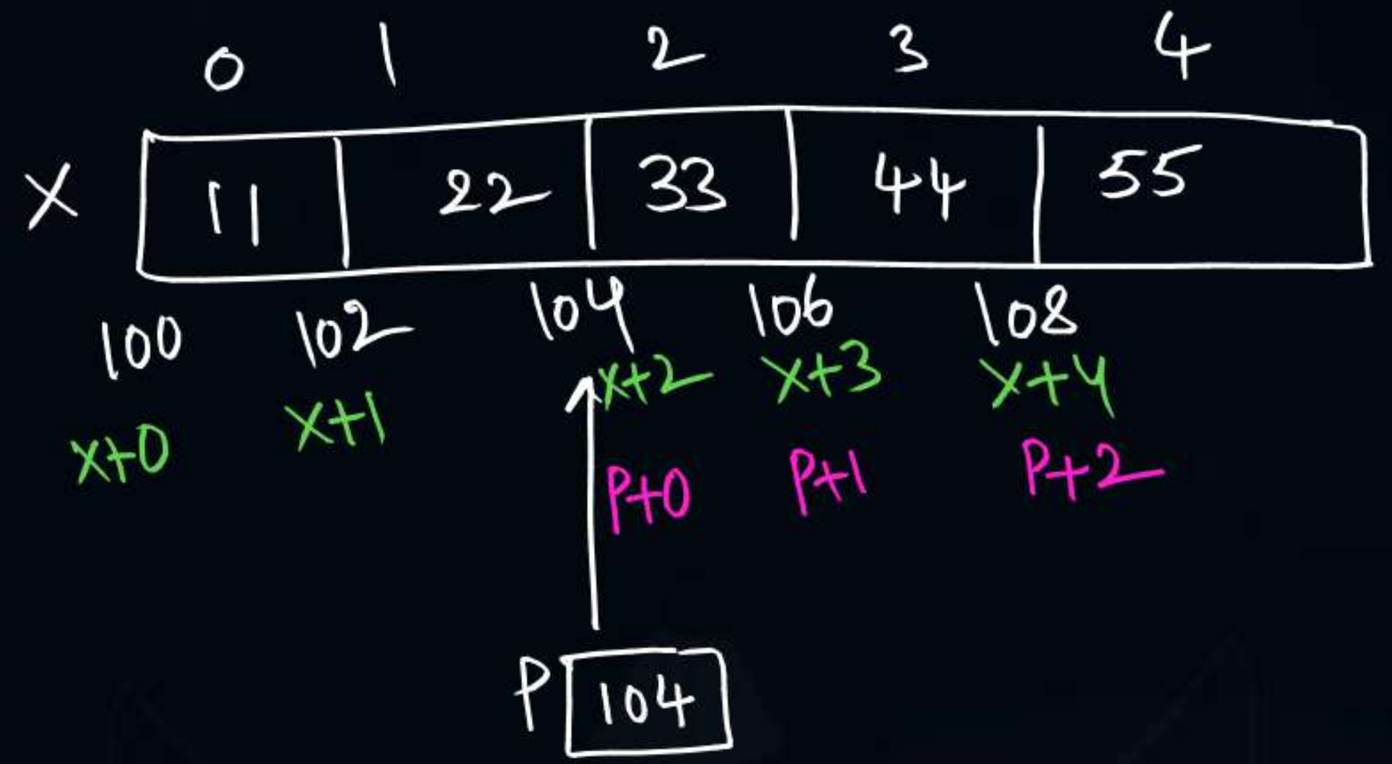
Topic : Pointers and Arrays

Ex:

int x[5] = {11, 22, 33, 44, 55};

int *p = &x[2];

// NOTE: Pointer can hold address of any Element of array. But array always hold Base address.



$\boxed{x + i == \&x[i]}$ True always.

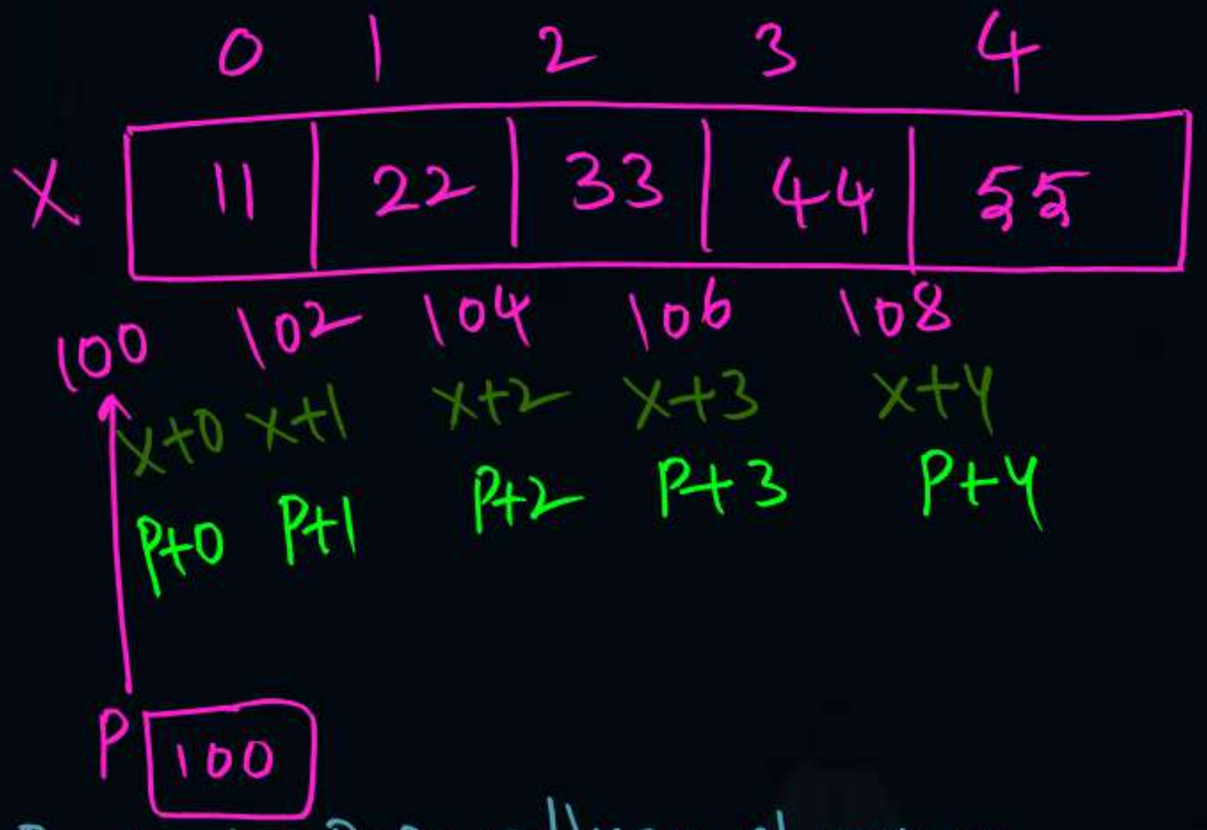
Now $x + i \neq p + i$

$\boxed{p + i == \text{address of next } i^{th} \text{ Element from } p.}$

Topic : Pointers and Arrays

```c
int x[5]={11,22,33,44,55};

int *P=x;  // P Pointed to Base addren
```



$*(x+0) \Rightarrow$ value at addren$(x+0)$

$== x[0]$

$*(x+2) == x[2]$

$*(x+i) == x[i]$

$*(P+0) == x[0]$

$*(P+1) == P[1]$

$\vdots$

$*(P+i) == P[i]$

When Pointer Points to Base addren of array.

$x+i == P+i == \&x[i]$

$*(x+i) == *(P+i) == x[i] == P[i]$

$i[x] == x[i]$

$i[P] == P[i]$

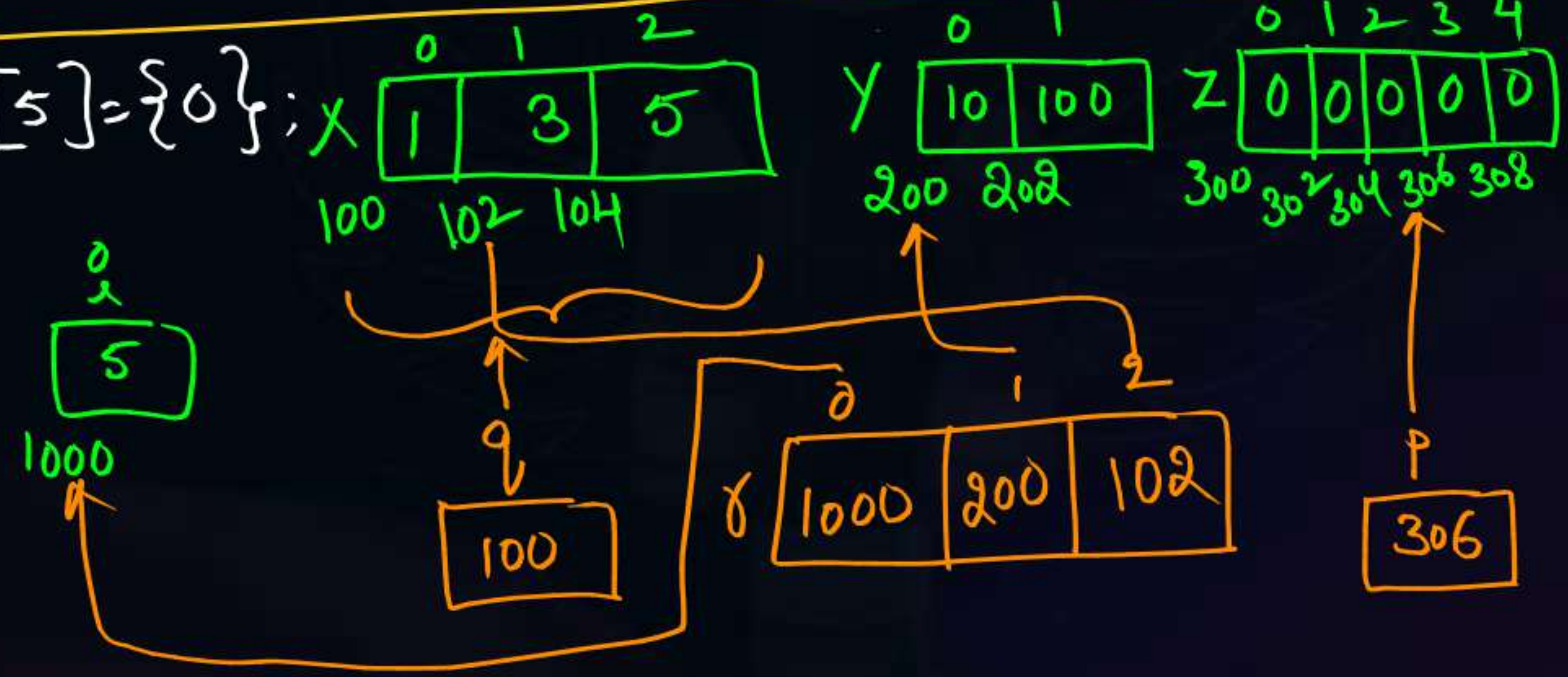
Topic : Pointers and Arrays

Pointers can be Used with arrays in either of 3 ways:

		Ex:
(1) Pointer to an Element of array	datatype *Pointer;	int *P;
(2) Pointer to whole array [Pointer array (or) array Pointer]	datatype (*Pointer)[size];	int (*P)[5];
(3) Array of Pointers	datatype *Pointer[size];	int *P[5];

Ex:

```
int x[3]={1,3,5}, Y[2]={10,100}, Z[5]={0};
int *p, (*q)[3], *r[3]; i=5;
p=&Z[3];     | r[0]=&i;
q=&x;        | r[1]=Y;
             | r[2]=&x[1];
```



Topic : Pointers and Arrays

Pointers and 2-D arrays

$*(x+0)=100$

$*(x+1)=116$

$*(x+2)=132$

$\boxed{*(x+i)=\&x[i][\ ]}$

$(x+0)+2 = 100+2*4 = 108$

$(x+1)+1 = 116+1*4 = 120$

$(x+2)+3 = 132+3*4 = 144$

$\boxed{(x+i)+j = \&x[i][j]}$

int x[3][4] = {1, 3, 5, 7, 2, 4, 6, 8, 11, 15, 19, 24};

Let B=100

1 int = 4 Bytes

x	0	1	2	3
p = *(x+0) → 0, a = x+0	100: 1	104: 3	108: (x+0)+2, 5	112: 7
q = *(x+1) → 1, b = x+1	116: 2	120: (x+1)+1, 4	124: 6	128: 8
r = *(x+2) → 2, c = x+2	132: 11	136: 15	140: 19	144: (x+2)+3, 24

$x+0 \Rightarrow 100$

$x+1 \Rightarrow 116$

$x+2 \Rightarrow 132$

$\boxed{x+i = \&x[i][\ ]}$

Address of Row i

Row 0 is a 1-D array, with name x+0

Row 1 is a 1-D array, with name = x+1

Row 2 is a 1-D array with name = x+2

2 mins Summary

$$(x+0)+1 \Rightarrow \&x[0][1] = 104$$

$$*(x+0)+1 \Rightarrow \&x[0][1] = 104$$

$$*(*(x+0)+1) = x[0][1] = 3$$

$$*(*(x+2)+3) = x[2][3] = 24$$

$$\boxed{*(*(x+i)+j) = x[i][j]}$$

- Pointers vs Arrays
- Pointer to Element
- Pointer to whole array
- Array of Pointers
- Pointer with 1-D array
- Pointer with 2-D array

THANK - YOU